



TRANSFORMACIÓN DIGITAL

Periodo: Del 29 de junio al 03 de julio de 2026.

Duración: 10 hrs.

Horario: 10:00 a 12:00 hrs.

Costo UNAM: \$600.00

Público en general: \$1, 000.00

Curso: Python Avanzado

Objetivo del curso:

El objetivo de este curso es dominar Python más allá de conocer su sintaxis; comprender la filosofía que lo hace una de las herramientas más potentes en la ingeniería de software moderna. Este curso está diseñado como una inmersión profunda en las entrañas del lenguaje, donde dejaremos atrás los scripts lineales para explorar la elegancia de las funciones de orden superior, la sofisticación de los decoradores y la robustez de la programación orientada a objetos avanzada. A lo largo de nuestras cinco sesiones, veremos el funcionamiento del modelo de objetos de Python, aprenderemos a gestionar el flujo de datos de manera eficiente mediante generadores y enfrentaremos el reto de la concurrencia y la asincronía para maximizar el rendimiento de nuestras aplicaciones. No se trata simplemente de aprender nuevas librerías, sino de transformar nuestra mentalidad de programadores para escribir código "Pythonic": aquel que no solo es funcional, sino que es legible, mantenible y capaz de escalar ante las demandas de entornos de producción reales. Es el puente definitivo entre ser un usuario de Python y convertirse en un desarrollador experto en su ecosistema.

Algunos objetivos específicos son:

1. **Programación Funcional:** Comprender y aplicar funciones de orden superior, generadores y comprensiones para escribir código más compacto y eficiente.
2. **Metaprogramación y Decoradores:** Aprender a crear decoradores personalizados para modificar el comportamiento de funciones y clases de manera dinámica.
3. **Programación Orientada a Objetos Pro:** Dominar conceptos avanzados como la herencia múltiple, el algoritmo MRO y la sobrecarga de operadores mediante métodos mágicos.
4. **Gestión de Atributos:** Utilizar descriptores y propiedades para controlar el acceso y la validación de datos dentro de las estructuras de objetos.

5. **Programación Asíncrona:** Introducir los conceptos de `asyncio` y tareas concurrentes para optimizar el rendimiento en aplicaciones con alta carga de entrada y salida de datos.

Requerimientos del curso:

-Equipo con mínimo un procesador multinúcleo (Intel Core i3/Ryzen 3 de generaciones recientes) y RAM de 16 gb de preferencia

Introducción

Este curso es una inmersión técnica de 10 horas diseñada para transformar tu capacidad de desarrollo, evolucionando de la escritura de scripts básicos hacia la creación de software profesional, eficiente y escalable. A lo largo de cinco sesiones prácticas, cubriremos los siguientes pilares:

- **Optimización y Código "Pythonic":** Aprenderás a utilizar funciones de orden superior, generadores y herramientas de `itertools` para procesar datos de manera elegante y con un uso mínimo de memoria.
- **Metaprogramación y Extensibilidad:** Dominarás el uso de clausuras y la creación de decoradores personalizados para añadir funcionalidades avanzadas a tus funciones y clases de forma dinámica.
- **Arquitectura de Objetos Avanzada:** Profundizaremos en la herencia múltiple, el algoritmo MRO y el uso de métodos "dunder" para integrar tus objetos de forma nativa con el lenguaje, incluyendo la gestión de contextos con `with`.
- **Integridad y Control de Datos:** Implementarás validaciones automáticas y control de acceso a atributos mediante el uso de propiedades y descriptores de clase.
- **Rendimiento y Asincronía:** Entenderás las limitaciones del GIL y aprenderás a ejecutar tareas concurrentes utilizando `asyncio` para maximizar la velocidad de tus aplicaciones en entornos de alta demanda

Contenido del curso:

Módulo 1: Pythonic Code y Funciones de Orden Superior

- Lección 1.1: Sintaxis avanzada. Comprensión de sintaxis avanzada en listas, diccionarios y conjuntos.
- Lección 1.2: Generadores. Se verá el uso de `yield` y expresiones generadores para lograr manejar gran masa de datos sin saturar la RAM.
- Lección 1.3: Manipulación de iteradores para la eficiencia. Se implementará `map`, `filter` y `reduce`, junto con el módulo `itertools` para la manipulación de iteradores.

Módulo 2: Clausuras y Decoradores

- Lección 2.1: Regla LEGB. Comprensión de ámbitos Local, Enclosing, Global y Built-in.
- Lección 2.2: Clausuras. Captura del estado de su entorno.
- Lección 2.3: Manejo de Argumentos. Uso de *args y **kwargs, así como también de functools.wraps.

Módulo 3: POO Avanzada y Métodos.

- Lección 3.1: Herencia múltiple. Comprensión de la jerarquía de clases en Python.
- Lección 3.2: Métodos Dunder. Personalización del ciclo de vida de un objeto.
- Lección 3.3: Gestión de contexto. Manejo de __enter__ y __exit__ para manejar de forma segura los recursos con with.

Módulo 4: Metaprogramación y Descriptores

- Lección 4.1: Encapsulamiento. Uso de @property, .getter, .setter y .deleter.
- Lección 4.2: Métodos de clase y estáticos. Diferencias de uso entre @classmethod y @staticmethod para lógica de fábrica y utilidades.
- Lección 4.3: Descriptores de acceso. Creación de lógica de validación reutilizable.

Módulo 5: Concurrencia y Programación Asíncrona

- Lección 5.1: GIL. Comprensión del concepto Global Interpreter Lock e hilos.
- Lección 5.2: Asyncio. Comprensión de la sintaxis de async y await, event loop y gestión de corrutinas.
- Lección 5.3: Concurrencia vs Paralelismo. Comprensión de diferencias entre threading, multiprocessing y asyncio en tareas.

Recursos y Materiales

Visual Studio Code con la extensión de Python

Evaluación

Actividades	30%
Evaluaciones (diarias)	20%
Proyecto	50%

Perfil de ingreso:

- Dominio de sintaxis básica: Capacidad para escribir scripts que utilicen estructuras de control (if, for, while) y tipos de datos fundamentales (listas, diccionarios, tuplas).
- Manejo de funciones: Comprensión clara de la definición de funciones, paso de argumentos y retorno de valores.
- Fundamentos de POO: Conocimiento básico de qué es una clase, un objeto y el uso del método `init`.
- Uso del entorno: Familiaridad con el uso de la terminal o línea de comandos y gestión básica de archivos.
- Pensamiento algorítmico: Capacidad para resolver problemas lógicos mediante la descomposición de tareas.

Perfil de egreso:

- **Escribir Código "Pythonic":** Implementar soluciones elegantes utilizando generadores y funciones de orden superior que optimicen el uso de memoria y la legibilidad.
- **Dominar la Extensibilidad:** Crear y aplicar decoradores avanzados para añadir funcionalidades transversales (como logging o validación) a sus aplicaciones de forma limpia.
- **Diseñar Arquitecturas Robustas:** Construir sistemas orientados a objetos complejos utilizando herencia múltiple, métodos mágicos y gestores de contexto personalizados.
- **Garantizar la Integridad de Datos:** Implementar validaciones automáticas y control de acceso a atributos mediante el uso de propiedades y descriptores de clase.
- **Gestionar el Alto Rendimiento:** Identificar cuellos de botella en aplicaciones y aplicar programación asíncrona con *asyncio* para mejorar la eficiencia en tareas de entrada/salida.
- **Aplicar Estándares Profesionales:** Desarrollar software bajo las recomendaciones de la guía de estilo PEP 8, facilitando el mantenimiento y la colaboración en proyectos de gran escala.

Modalidad del curso: *Presencial*

Duración del curso: *5 sesiones de 2 horas cada una.*